Project Repository
github.com/aacaas5/EN3150_A03

University of Moratuwa

EN3150 Assignment 03

Resource-Constrained CNN for Edge Image Classification

Module: EN3150

Group name: Feature Four

Group member	Index number
ARUNIYA R.	230059T
MUHAMATH M.A.A.A.	230416L
PIRIYATHARSAN M.	230491J
SNEKAN S.	230620G

Experimental study and reproducible implementation

Abstract

Four CNNs are evaluated on a deterministic stratified 70/15/15 split of 27,000 EuroSAT RGB images, using 64×64 inputs. Model A uses standard convolutions; Model B replaces later convolutions with depthwise-separable blocks. A controlled optimizer comparison selects Adam, and both custom models are trained for 25 epochs. Model B has 22,426 trainable parameters and achieves 94.15% test accuracy, compared with 94.30% for Model A. ImageNet-pretrained MobileNetV2 and EfficientNet-B0 achieve 97.31% and 96.81%, respectively, after two-stage fine-tuning. Model B reduces parameters by 86.41% relative to Model A, but does not improve its measured host latency. The results show a storage–accuracy trade-off and demonstrate why arithmetic counts alone cannot predict runtime.

Contents

1 Introduction **3**

2 Dataset and Data Preparation **3**

2.1 EuroSAT Dataset . . . 3
2.2 Dataset Classes . . . 3
2.3 64x64 Input . . . 3
2.4 70/15/15 Split . . . 3
2.5 Preprocessing and Augmentation . . . 4

3 Custom CNN Architectures **4**

3.1 Model A . . . 4
3.2 Model B . . . 4
3.3 Depthwise Separable Convolution . . . 5
3.4 Parameter Calculation . . . 5
3.5 Hardware-Aware Activation Selection . . . 5

4 Optimizer Selection and Tuning **6**

4.1 SGD . . . 6
4.2 SGD with Momentum . . . 6
4.3 Adam . . . 6
4.4 Experimental Comparison . . . 6
4.5 Selected Optimizer . . . 7

5 Custom CNN Training and Evaluation **7**

5.1 Training Setup . . . 7
5.2 Training Curves . . . 7
5.3 Test Results . . . 8
5.4 Confusion Matrices . . . 9
5.5 Model A vs Model B Resource Comparison . . . 11

6 Lightweight State-of-the-Art Models **11**
6.1 MobileNetV2 . . . 11
6.2 EfficientNet-B0 . . . 11
6.3 Transfer Learning Strategy . . . 11
6.4 Results . . . 12

7 Final Comparison **14**
7.1 Accuracy . . . 16
7.2 Parameter Count . . . 16
7.3 Memory Footprint . . . 16
7.4 Computational Cost . . . 16
7.5 Edge Deployment Trade-offs . . . 17

8 Discussion **18**

9 Conclusion **19**

A Layer-wise Architecture Tables **21**
A.1 Model A . . . 21
A.2 Model B . . . 22

B Reproducibility **22**

1 Introduction

CNNs learn local spatial features using shared convolution kernels [4]. Edge image classification requires a balance between recognition quality, stored weights, computation and measured execution time. This study compares a standard CNN (Model A), a custom depthwise-separable CNN (Model B), and two ImageNet-pretrained lightweight networks on one fixed EuroSAT split. The central constraint is fewer than 100,000 trainable parameters for Model B. Experiments use PyTorch [5]; no test samples influence optimizer selection, training, or checkpoint selection. The goal is to determine which resources the custom sub-100k model saves and what accuracy cost is observed, then compare that trade-off with pretrained MobileNetV2 and EfficientNet-B0.

2 Dataset and Data Preparation

2.1 EuroSAT Dataset

EuroSAT comprises Sentinel-2 satellite scenes for land-use and land-cover classification [1]. We use the RGB version exposed by `torchvision.datasets.EuroSAT`, preserving its original class labels.

2.2 Dataset Classes

Table 1 lists all ten original classes and their sample counts. Class sizes differ, so macro precision and recall complement overall accuracy by giving equal weight to each class. EuroSAT is the only dataset used for task-specific training and testing; CIFAR-10 is not used.

Table 1: EuroSAT class counts and the shared stratified allocation.

Class	Total	Train	Validation	Test
AnnualCrop	3000	2100	450	450
Forest	3000	2100	450	450
HerbaceousVegetation	3000	2100	450	450
Highway	2500	1750	375	375
Industrial	2500	1750	375	375
Pasture	2000	1400	300	300
PermanentCrop	2500	1750	375	375
Residential	3000	2100	450	450
River	2500	1750	375	375
SeaLake	3000	2100	450	450
Total	27000	18900	4050	4050

2.3 64x64 Input

All networks receive original 64×64 RGB images. The pretrained networks support this size through adaptive pooling; their usual larger ImageNet input recipe is not used. No images are upscaled above the assignment resolution cap.

2.4 70/15/15 Split

One stratified split is constructed with seed 3150: training receives 70%, and the held-out 30% is divided equally into validation and test sets. This gives 18,900 training, 4,050 validation and 4,050 test images, as shown in Table 1. Actual sample indices are saved as `train_indices.json`,

`val_indices.json` and `test_indices.json`, alongside a path/label manifest in `data/splits/`. Hash checks bind every run to exactly the same split and sample order. An exhaustive content audit found no pixel-identical images across sets. This is an image-level split; geographic independence is not established.

2.5 Preprocessing and Augmentation

Images are decoded into a reusable unsigned-byte cache, converted to floating point and divided by 255. Fixed ImageNet normalization uses means $(0.485, 0.456, 0.406)$ and standard deviations $(0.229, 0.224, 0.225)$. Independent horizontal flips with probability 0.5 apply only during training. Validation and test processing is deterministic. Custom and pretrained models use the same pipeline here; neither normalization nor augmentation changes sample membership or estimates statistics from held-out data.

3 Custom CNN Architectures

3.1 Model A

Four standard 3×3 convolutions use channel widths 24, 48, 96 and 128. Each is followed by batch normalization, ReLU and 2×2 max-pooling. Padding preserves convolutional spatial size. Global average pooling and a small linear head avoid a large flattened dense layer. Batch normalization provides learned channel scaling and stabilizes intermediate feature scales. The early feature maps are large, so the first stage uses only 24 channels. The width increases as pooling reduces the spatial size. This adds feature capacity without paying the cost of 128 channels at full 64×64 resolution. The final global average pool keeps the classifier small. Table 14 gives every layer's input/output shape, kernel, stride, padding, activation and parameter count. The total is 164,962; all parameters are trainable. The linear head outputs ten unnormalized logits for cross-entropy loss.

3.2 Model B

The stem, channel widths, pooling schedule and head match Model A. Each later standard convolution is replaced by a depthwise 3×3 convolution followed by a pointwise 1×1 convolution. Depthwise groups equal input channels. Keeping the same 24, 48, 96 and 128 widths and the same pooling locations makes the custom comparison easier to interpret: both networks expose features at the same spatial scales and use the same-size classifier head. Each component has batch normalization and ReLU. These extra operations mean this is not a strict single-operator ablation. Table 15 identifies depthwise and pointwise operations separately. Channel counts are the second dimension of each NCHW shape, and activations are shown as separate rows. Model B is built from scratch using a reusable `DepthwiseSeparableConv` block.

The executed count is 22,426 trainable parameters, leaving 77,574 below the 100,000 limit. The constructor and verification script check this programmatically:

```
count = sum(p.numel() for p in model.parameters()
            if p.requires_grad)
assert count < 100000
```

The sum of the layer counts in Table 15 independently agrees with this result.

3.3 Depthwise Separable Convolution

Depthwise filtering processes channels independently; pointwise convolution learns cross-channel combinations [2]. For output spatial size $H_o \times W_o$, multiplying the weight expressions below by $H_o W_o$ gives the corresponding convolution MAC counts for one image. Here a MAC is one multiply–accumulate operation. The factorization reduces the number of weights because a separate spatial kernel is no longer learned for every input–output channel pair. It also restricts the possible filters, so equal accuracy is not guaranteed.

3.4 Parameter Calculation

With convolution biases disabled,

$$P_{\text{standard}} = K_h K_w C_{\text{in}} C_{\text{out}}, \tag{1}$$

$$P_{\text{DSC}} = K_h K_w C_{\text{in}} + C_{\text{in}} C_{\text{out}}. \tag{2}$$

In Equations (1) and (2), K_h, K_w are kernel dimensions and $C_{\text{in}}, C_{\text{out}}$ are channel counts. Dividing the two expressions gives

$$\frac{P_{\text{DSC}}}{P_{\text{standard}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K_h K_w}, \qquad \text{reduction} = 100\left(1 - \frac{P_{\text{DSC}}}{P_{\text{standard}}}\right)\%. \tag{3}$$

Table 2 applies Equation (3) to the three replaced convolutions. Batch normalization adds two trainable scalars per output channel; running statistics are non-trainable buffers. The classification head includes bias. Thus whole-network counts include more than the convolution weights in this table.

Table 2: Convolution weight counts excluding bias and normalization; reduction compares depthwise plus pointwise with standard convolution.

Channels	Standard	Depthwise	Pointwise	Reduction (%)
24 to 48	10368	216	1152	86.81
48 to 96	41472	432	4608	87.85
96 to 128	110592	864	12288	88.11

Summing the executed architecture tables gives 163,080 convolution weights, 592 normalization parameters and 1,290 head parameters for Model A, totalling 164,962. The corresponding Model B counts are 20,208, 928 and 1,290, totalling 22,426. Whole-network savings differ from an individual convolution's savings because the stem and head are retained and Model B adds normalization.

3.5 Hardware-Aware Activation Selection

ReLU computes $\max(0, x)$ using a simple comparison and avoids exponentials. It is widely supported by inference kernels and is compatible with common integer inference workflows. No quantization or activation-specific energy measurement is performed here, so hardware savings are not claimed empirically. ReLU is the deliberate choice for both custom networks. The pretrained architectures retain their original activations: MobileNetV2 uses ReLU6, whereas EfficientNet-B0 uses SiLU and sigmoid operations. They are not modified to match the custom networks.

4 Optimizer Selection and Tuning

4.1 SGD

Standard SGD updates parameters with the current minibatch gradient. The baseline uses learning rate 0.01 and no momentum.

4.2 SGD with Momentum

Momentum accumulates a velocity from earlier gradients; this can accelerate consistent descent directions [7]. The experiment uses learning rate 0.01 and momentum 0.9, without Nesterov acceleration. For minibatch gradient g_t, the update can be written as

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t, \tag{4}$$

where η is the learning rate and μ is the momentum coefficient. Equation (4) reduces to standard SGD when $\mu = 0$. Accumulating gradients can smooth inconsistent directions, although it does not guarantee a smoother validation curve.

4.3 Adam

Adam adapts updates using first and second gradient moments [3]. The tested learning rate is 0.001, with default betas $(0.9, 0.999)$ and $\epsilon = 10^{-8}$.

4.4 Experimental Comparison

Model B is used for its low compute requirement. Each recipe uses ten epochs, batch size 128, identical initial weights, seed, data order and augmentation strategy. No weight decay, scheduler or early stopping is used. These are controlled practical recipe comparisons, not exhaustive learning-rate searches. SGD and momentum SGD share learning rate 0.01, while Adam uses 0.001. The experiment therefore supports the selected configuration for this project; it is not an identical-learning-rate comparison of optimizer algorithms. Table 3 records settings, best validation performance and final validation accuracy; Table 4 adds final losses and timing. "Final" here means the last pilot epoch, not a test-set result. The pilot test sets are not evaluated for optimizer selection.

Table 3: Optimizer comparison on Model B; each run uses ten epochs. Adam moment coefficients are specified in the text.

Optimizer	LR	Momentum	Best val. (%)	Epoch	Final val. (%)
Adam	0.001	–	91.46	10	91.46
SGD + momentum	0.01	0.9	90.57	8	90.52
SGD	0.01	0	81.63	9	80.05

Table 4: Final-epoch optimizer losses and training accuracy; mean timing spans all ten epochs.

Optimizer	Train loss	Val. loss	Train acc. (%)	Epoch (s)
SGD	0.545	0.573	81.52	3.164
SGD + momentum	0.250	0.277	91.35	2.963
Adam	0.220	0.246	92.30	3.078

Figure 1 shows that plain SGD improves gradually but finishes with higher validation loss than the other recipes. Momentum reaches a much higher validation accuracy within the same budget, despite fluctuations during the middle epochs. Adam finishes with the lowest validation loss, 0.246, compared with 0.277 for momentum SGD and 0.573 for plain SGD.

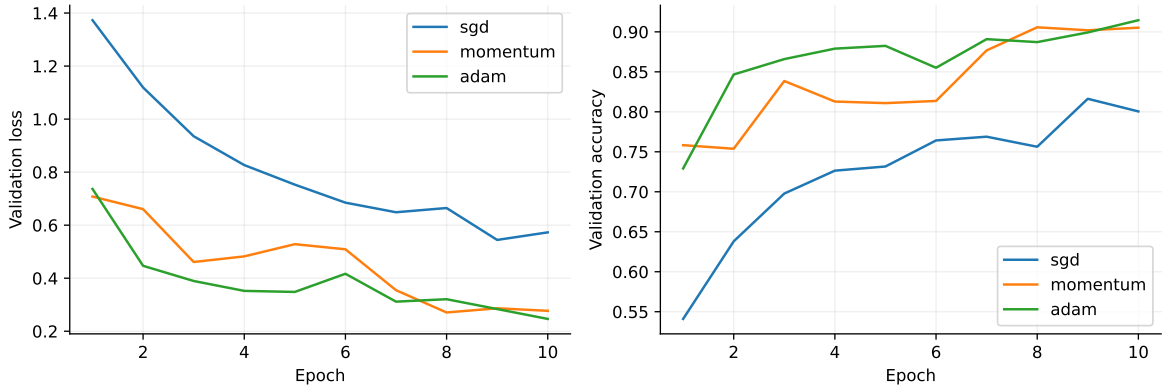


Figure 1: Validation loss and accuracy; unsmoothed observations.

4.5 Selected Optimizer

Adam is selected by highest validation accuracy, with lower validation loss breaking ties. Its best validation accuracy is 91.46%, compared with 90.57% for momentum SGD and 81.63% for plain SGD. Momentum therefore improves on plain SGD by 8.94 percentage points at the same learning rate. Adam's advantage over momentum SGD is smaller, 0.89 percentage points, and also involves a different learning rate. This experiment supports choosing Adam for the final runs; it does not isolate optimizer type from learning-rate choice or establish universal superiority. Losses are still improving near the end, so the fixed budget does not establish full convergence. The initial settings were retained, and no setting was chosen using test performance.

5 Custom CNN Training and Evaluation

5.1 Training Setup

Both custom models are initialized afresh and trained for 25 epochs using the selected optimizer Adam at learning rate 0.001, batch size 128 and cross-entropy loss. Thus both exceed the required minimum of 20 epochs. Computation uses 32-bit floating point (FP32) on an NVIDIA GeForce RTX 5060 Laptop GPU. Deterministic algorithms and seed 3150 are enabled. The best validation-accuracy checkpoint, with validation loss breaking ties, is retained. There is no early stopping. The first ten epochs of the final Model B run exactly reproduce its Adam pilot losses and accuracies. One reusable training loop handles the custom and pretrained models, with configurable epochs, batch size and optimizer, CUDA support and CPU fallback. Python 3.14.5, PyTorch 2.13.0+cu130 and torchvision 0.28.0+cu130 are recorded in each run's JSON. CSV histories contain every epoch's losses, accuracies, stage and elapsed time. Checkpoints store the validation-selected weights, not the final epoch by default.

5.2 Training Curves

Figures 2 and 3 show the complete training histories. Both training losses decrease overall, while validation losses fluctuate. The gap between training and validation performance near the end

suggests that continued fitting of the training set does not reliably improve held-out performance.

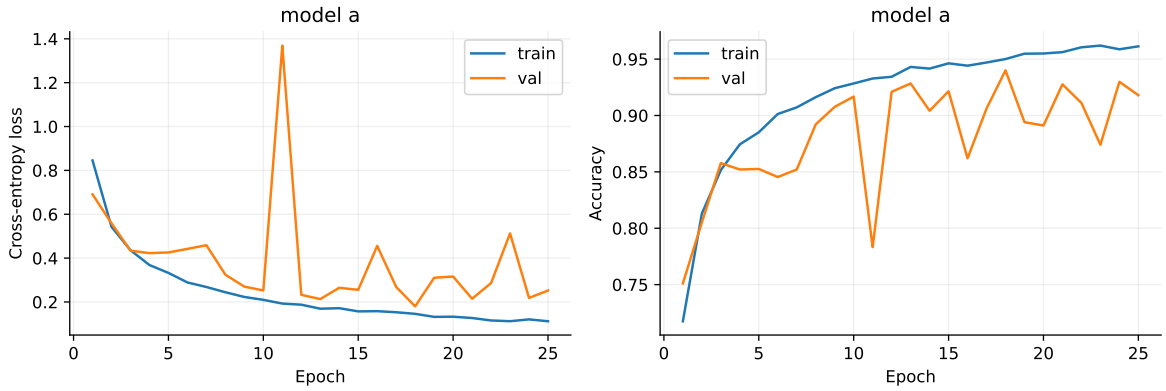


Figure 2: Model A training and validation history.

Model A selects epoch 18 with 94.00% validation accuracy. Final training/validation accuracies are 96.13%/91.80%. Its largest epoch-to-epoch validation drop is 13.36 percentage points at epoch 11, followed by recovery. This is a temporary instability in the observed validation trajectory; the experiment does not identify its cause.

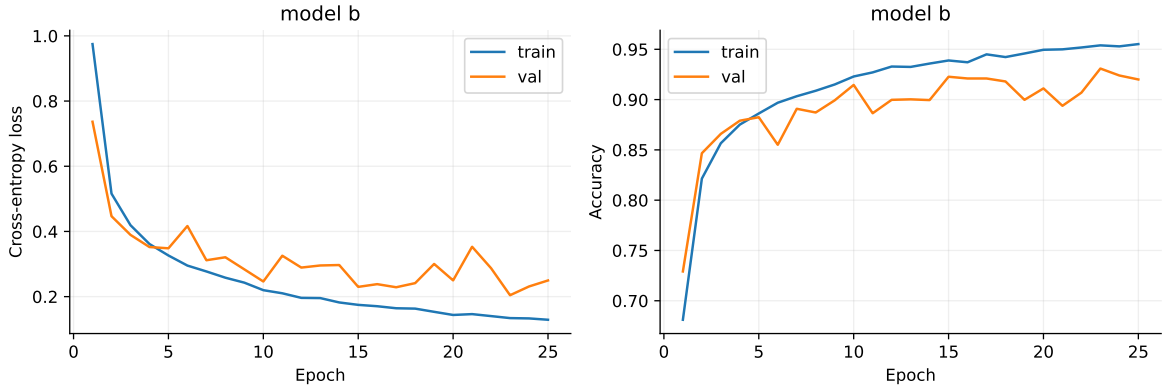


Figure 3: Model B training and validation history.

Model B selects epoch 23 with 93.09% validation accuracy. Final training/validation accuracies are 95.52%/92.00%. Its largest epoch-to-epoch validation drop is 2.81 percentage points at epoch 11, smaller than Model A's largest drop. However, neither run gives a monotonic validation improvement, which supports retaining the best validation checkpoint rather than using the last epoch automatically.

5.3 Test Results

Accuracy is the fraction of correct predictions. Precision, recall and F1 are unweighted macro averages over all ten classes; undefined class precision is set to zero. Weighted versions are also retained in the machine-readable comparison files. Each selected checkpoint is evaluated on the untouched test set. For class c, let TP_c, FP_c and FN_c denote true positives, false positives and

false negatives. With $C = 10$ classes and $N = 4050$ test images,

$$\begin{aligned} \text{Accuracy} &= \frac{\sum_c TP_c}{N}, \\ \text{MacroPrecision} &= \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FP_c}, \\ \text{MacroRecall} &= \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FN_c}, \\ \text{MacroF1} &= \frac{1}{C}\sum_{c=1}^{C}\frac{2P_cR_c}{P_c + R_c}. \end{aligned} \tag{5}$$

The scikit-learn calculations in Table 5 follow Equation (5). Model A is slightly higher on all three aggregate metrics, but its accuracy advantage corresponds to only 6 additional correct predictions on this test set. This is a small observed difference, not evidence of a consistent advantage over repeated runs.

Table 5: Held-out test metrics for custom models; P is precision and R is recall.

Model	Accuracy (%)	Macro P (%)	Macro R (%)
Model A	94.30	94.25	94.26
Model B	94.15	94.00	93.88

5.4 Confusion Matrices

Figures 4 and 5 show raw counts, not row percentages. Model A confuses HerbaceousVegetation with PermanentCrop more often, while Model B's largest directed error is Highway predicted as Industrial. Similar overall accuracy therefore does not imply identical class-level behaviour; Section 8 examines these errors.

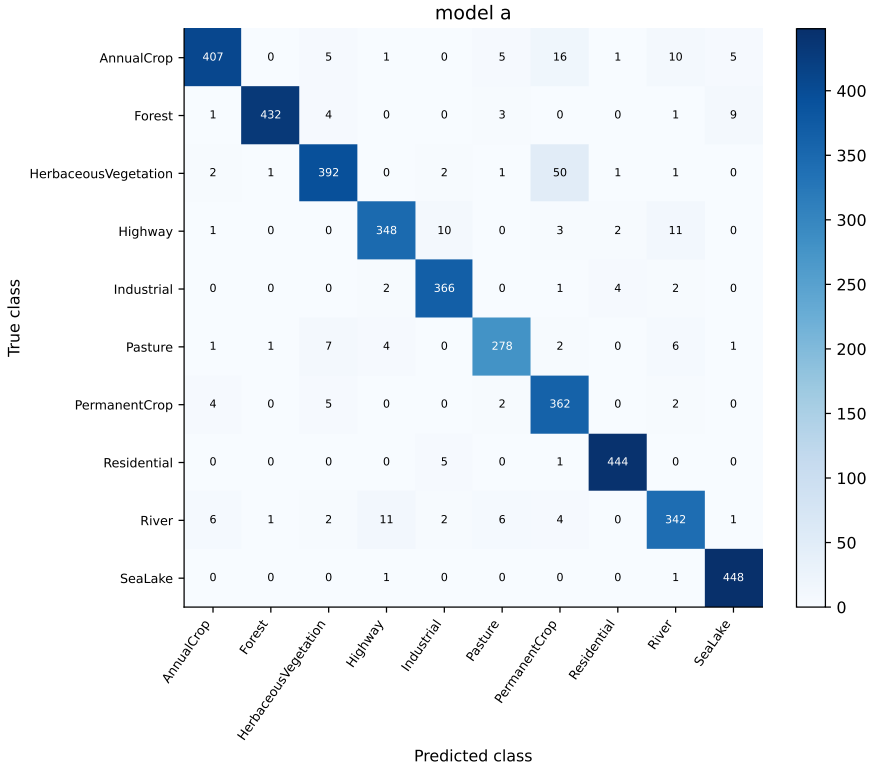


Figure 4: Model A: numeric counts, true classes on rows and predictions on columns.

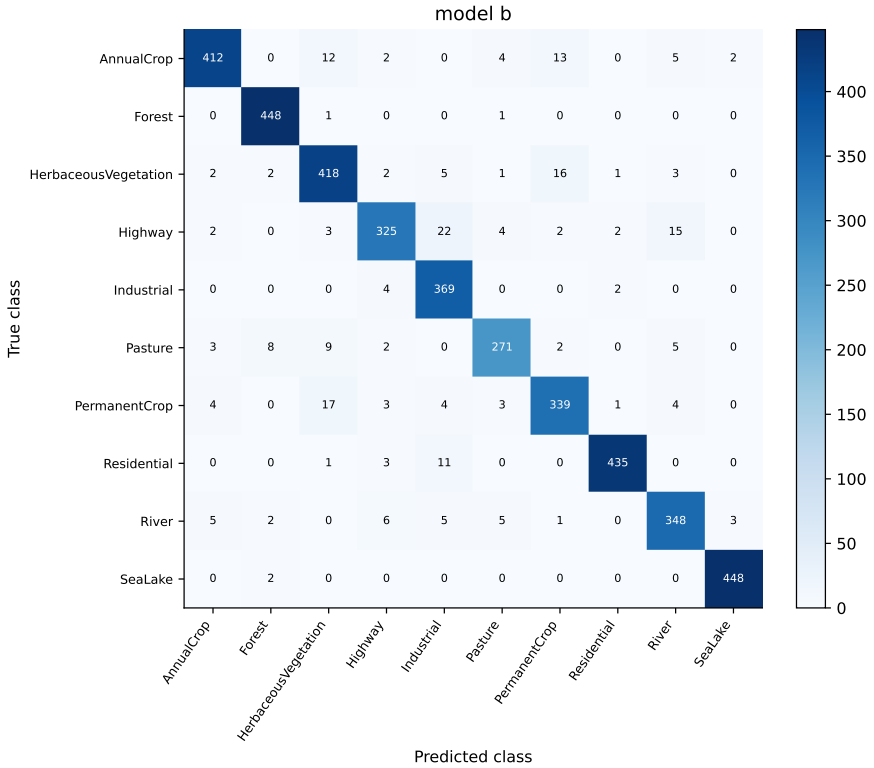


Figure 5: Model B: numeric counts, true classes on rows and predictions on columns.

5.5 Model A vs Model B Resource Comparison

Table 6 compares measured checkpoint sizes and mean epoch times with parameter and MAC counts. Timing includes training, validation and batch preprocessing, with CUDA synchronization at the timing boundaries; it excludes checkpoint and CSV writes. It is therefore a mean complete-epoch time, not a training-only kernel benchmark.

Table 6: Custom-model resources. All parameters are trainable; epoch time includes validation.

Model	Parameters	Size (KiB)	Epoch (s)	MACs (M)
Model A	164,962	654.46	3.246	30.967
Model B	22,426	104.09	3.646	6.188

Model B reduces whole-network parameters by 86.41%, serialized size by 84.09%, and counted MACs by 80.02%. Its test accuracy is 0.15 percentage points lower. The mean epoch time nevertheless rises from 3.246 s to 3.646 s. Depthwise kernel efficiency, additional normalization/activation operations and memory or launch overhead can prevent arithmetic savings from producing speedups. These are possible explanations; the timing experiment does not separate their contributions.

6 Lightweight State-of-the-Art Models

6.1 MobileNetV2

MobileNetV2 uses inverted residual blocks and linear bottlenecks [6]. We load torchvision's explicit ImageNet `IMAGENET1K_V2` weights and replace its classifier with a ten-class head. Its depthwise layers reduce spatial filtering cost, while pointwise expansion and projection layers mix channels. It is still much larger than Model B: "lightweight" does not imply compliance with the custom parameter limit.

6.2 EfficientNet-B0

EfficientNet balances network depth, width and resolution [8]. We use torchvision's B0 `IMAGENET1K_V1` weights and replace its final classifier. No fallback to random initialization is permitted. It retains inverted bottlenecks, squeeze-and-excitation channel weighting and its original nonlinearities. The pretrained comparison includes prior ImageNet learning as well as architectural differences.

6.3 Transfer Learning Strategy

Stage one trains the head for two epochs using Adam at 0.001; feature weights and feature batch-normalization running statistics remain frozen. Stage two unfreezes the entire network and uses a new Adam optimizer at 0.0001 for eight epochs. Validation selects the checkpoint across both stages. Both models use exactly the saved custom-model split and the same 64x64 preprocessing. Table 7 distinguishes trainable counts and timing in the frozen-head and full-network stages. Both use the shared training loop.

Table 7: Transfer-learning stages, learning rates and measured mean epoch times.

Model	Stage	Epochs	LR	Trainable	Epoch (s)
MobileNetV2	head	2	0.001	12,810	3.347
MobileNetV2	finetune	8	0.0001	2,236,682	7.915
EfficientNet-B0	head	2	0.001	12,810	4.263
EfficientNet-B0	finetune	8	0.0001	4,020,358	11.175

6.4 Results

Table 8 reports classification metrics, while Table 9 separates total and fine-tuning trainable parameters, checkpoint size and epoch time. MobileNetV2 is 0.49 percentage points more accurate than EfficientNet-B0 in this run despite having fewer parameters. A larger pretrained model does not automatically give a better result at this input size and training budget.

Table 8: Held-out test metrics for pretrained models; P is precision and R is recall.

Model	Accuracy (%)	Macro P (%)	Macro R (%)
MobileNetV2	97.31	97.20	97.21
EfficientNet-B0	96.81	96.72	96.71

Table 9: Pretrained-model resources. Trainable counts refer to fine-tuning; timing averages both stages.

Model	Total	Trainable	Size (MiB)	Epoch (s)
MobileNetV2	2,236,682	2,236,682	8.754	7.002
EfficientNet-B0	4,020,358	4,020,358	15.604	9.793

Figure 6 shows improvement from 90.84% validation accuracy after head training to a best of 97.51% at epoch 10, a gain of 6.67 percentage points. Updating the features is therefore useful under this schedule. Training loss increases at the first fine-tuning epoch, coinciding with unfreezing and switching the feature extractor to training mode. Feature batch-normalization statistics are then updated; EfficientNet-B0 also activates stochastic depth in its feature extractor. These changed training conditions must be considered when reading the stage-transition losses.

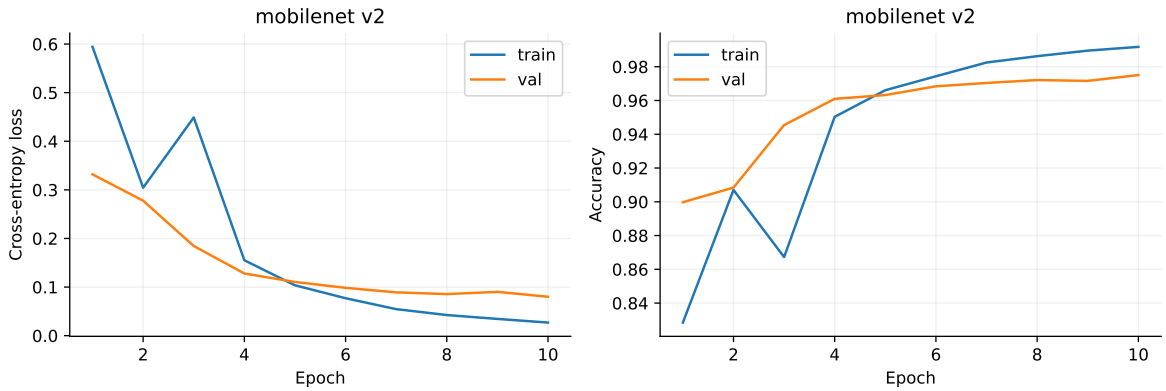


Figure 6: MobileNetV2: frozen-head epochs 1–2 and full-network fine-tuning epochs 3–10.

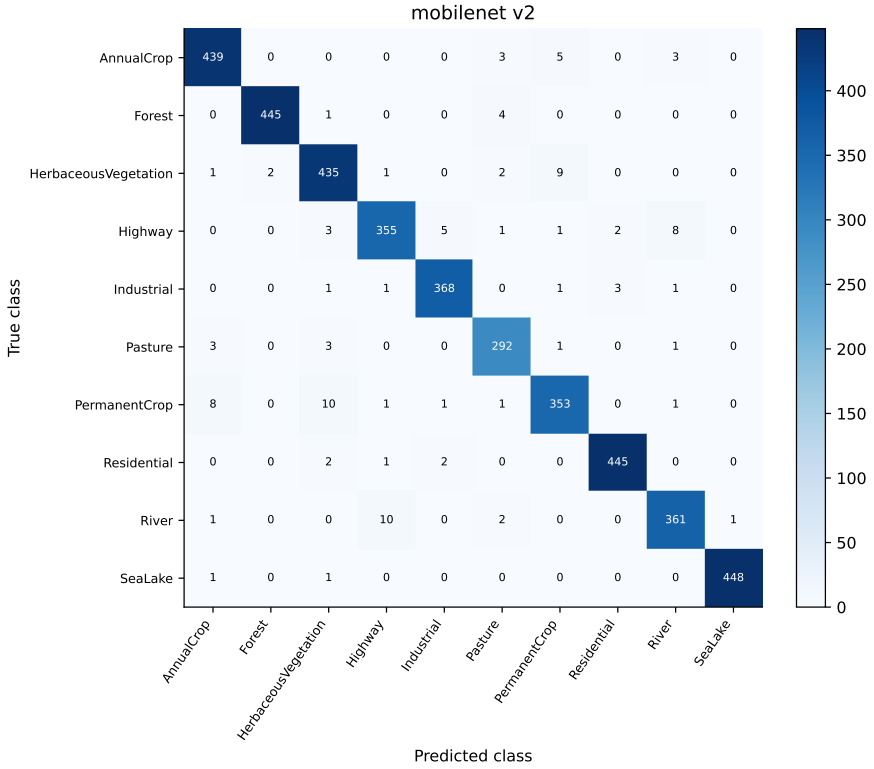


Figure 7: MobileNetV2 test confusion matrix (counts).

Figure 7 shows MobileNetV2's remaining class errors; Section 8 examines the largest directed confusions. EfficientNet-B0 improves from 89.78% validation accuracy after head training to 96.86% at epoch 8, a gain of 7.09 percentage points. Figure 8 shows the same stage-transition training-loss increase, followed by improvement. Training loss continues to fall late in the run while validation performance levels off, so its selected checkpoint precedes the final epoch. Figure 9 shows its test errors, including confusion between crop classes.

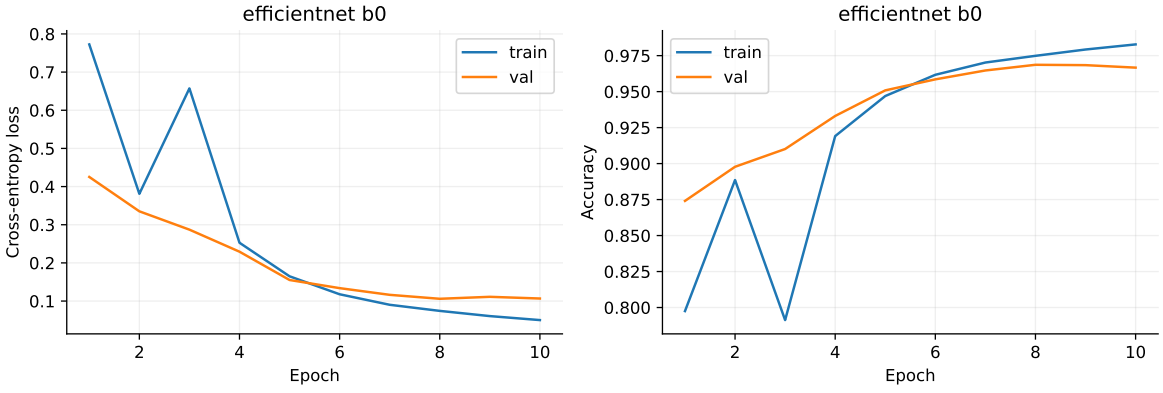


Figure 8: EfficientNet-B0: frozen-head epochs 1–2 and full-network fine-tuning epochs 3–10.

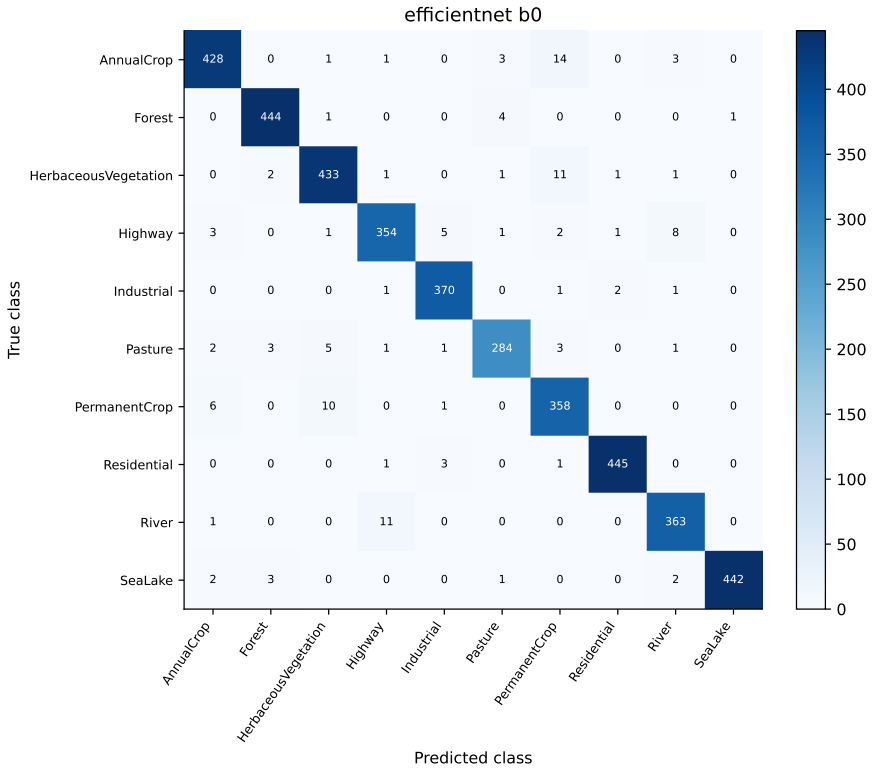


Figure 9: EfficientNet-B0 test confusion matrix (counts).

7 Final Comparison

Table 10 places the main accuracy and resource evidence in one location. Macro F1 and the weighted metrics in the accompanying CSV were recomputed from the saved test predictions; no model was retrained.

Table 10: Master held-out test and resource comparison. Size is the measured FP32 state-dictionary file. Latency is the original batch-one host benchmark.

Model	Acc. (%)	Macro P	Macro R	Macro F1	Params	Size (MiB)	MACs (M)	Mean (ms)	p95 (ms)
Model A	94.30	94.25	94.26	94.17	164,962	0.639	30.967	1.358	1.802
Model B	94.15	94.00	93.88	93.89	22,426	0.102	6.188	1.723	2.459
MobileNetV2	97.31	97.20	97.21	97.20	2,236,682	8.754	24.461	8.452	13.143
EfficientNet-B0	96.81	96.72	96.71	96.71	4,020,358	15.604	31.979	12.454	27.092

Table 11: Largest observed single intermediate activation tensor for batch-one FP32 inference. This is not total peak runtime RAM.

Model	Shape	Elements	FP32 (KiB)
Model A	1×24×64×64	98,304	384.0
Model B	1×24×64×64	98,304	384.0
MobileNetV2	1×96×32×32	98,304	384.0
EfficientNet-B0	1×96×32×32	98,304	384.0

7.1 Accuracy

Both pretrained models outperform the custom networks on this test set. For Model B, the question is whether that improvement justifies the additional storage and computation for the intended device. Relative to MobileNetV2, Model B gives 3.16 percentage points lower test accuracy; relative to EfficientNet-B0, the difference is 2.67 percentage points. These are observed accuracy costs under the tested configurations.

7.2 Parameter Count

The parameter totals count learned tensors only, while trainable totals reflect the active training stage. Frozen pretraining parameters still occupy storage. Tables 6 and 9 show 22,426 parameters for Model B, compared with 2,236,682 for MobileNetV2 and 4,020,358 for EfficientNet-B0. Model B saves 99.00% and 99.44%, respectively. Freezing feature weights reduces the trainable count but does not remove their storage or inference cost.

7.3 Memory Footprint

Model B occupies 0.102 MiB, compared with 8.754 MiB for MobileNetV2 and 15.604 MiB for EfficientNet-B0. These are reductions of 98.84% and 99.35%. Serialization and buffers explain why byte reductions differ from parameter reductions. Sizes include batch-normalization buffers and serialization overhead but exclude optimizer state. KiB and MiB use powers of 1024. The CSV column suffixes `kb/mb` use these binary units. Disk size is not peak inference RAM; it does not include every activation, framework allocation or operator workspace. Table 11 gives a narrower batch-one FP32 estimate: the largest single leaf-layer output tensor observed by hooks. All four maxima are 384 KiB, but this equality does not imply equal total peak RAM. Lifetimes, residual connections and temporary workspaces are not counted.

7.4 Computational Cost

Table 12 compares one-image arithmetic counts with measured CPU latency. Model B uses 74.70% fewer counted MACs than MobileNetV2 and 80.65% fewer than EfficientNet-B0. These savings concern Conv2d and Linear operations only.

Table 12: One 64x64 image: Conv/Linear arithmetic estimates and batch-one host CPU latency.

Model	MACs (M)	2xMAC (MFLOP)	CPU mean (ms)	CPU p95 (ms)
Model A	30.967	61.934	1.358	1.802
Model B	6.188	12.377	1.723	2.459
MobileNetV2	24.461	48.923	8.452	13.143
EfficientNet-B0	31.979	63.959	12.454	27.092

Table 13: Fresh CPU-only repeat of the original forward-pass benchmark. It is additional timing evidence and does not replace the original reported latency.

Model	Mean	Median	p95	Std. dev. (ms)
Model A	4.713	4.629	5.519	0.447
Model B	6.301	6.134	7.238	0.746
MobileNetV2	28.570	28.719	30.490	1.451
EfficientNet-B0	36.958	37.069	39.157	1.537

MAC accounting uses forward hooks to obtain executed layer output shapes, kernel sizes and group counts, avoiding an external profiling dependency. It excludes normalization, activations, pooling, residual additions and data movement; twice MACs is only an arithmetic FLOP estimate. CPU timing uses the Intel Core i7-13620H host, four PyTorch threads, FP32, batch one, 20 warm-ups and 100 timed forwards. It excludes preprocessing and file I/O. The preallocated benchmark input is a zero tensor of the required shape; this is a forward-pass microbenchmark, not end-to-end application latency. GPU epoch times and CPU inference times measure different workloads. In Table 12, Model B takes 1.723 ms on average versus 1.358 ms for Model A despite its lower MAC count. MobileNetV2 and EfficientNet-B0 take 8.452 ms and 12.454 ms, respectively.

The original timing samples were not saved, so their standard deviations cannot be reconstructed. Table 13 is a fresh CPU-only repeat using the same checkpoints, input shape, four threads, 20 warm-ups and 100 timed forwards. It is reported separately because background load and software state can change timing. The absolute values differ from the original run, but the ordering is unchanged and Model B remains slower than Model A.

Figure 10 combines accuracy with parameter and MAC cost. Model B occupies the low-resource corner. MobileNetV2 gives the best accuracy and uses fewer counted MACs than Model A, but it needs a much larger checkpoint.

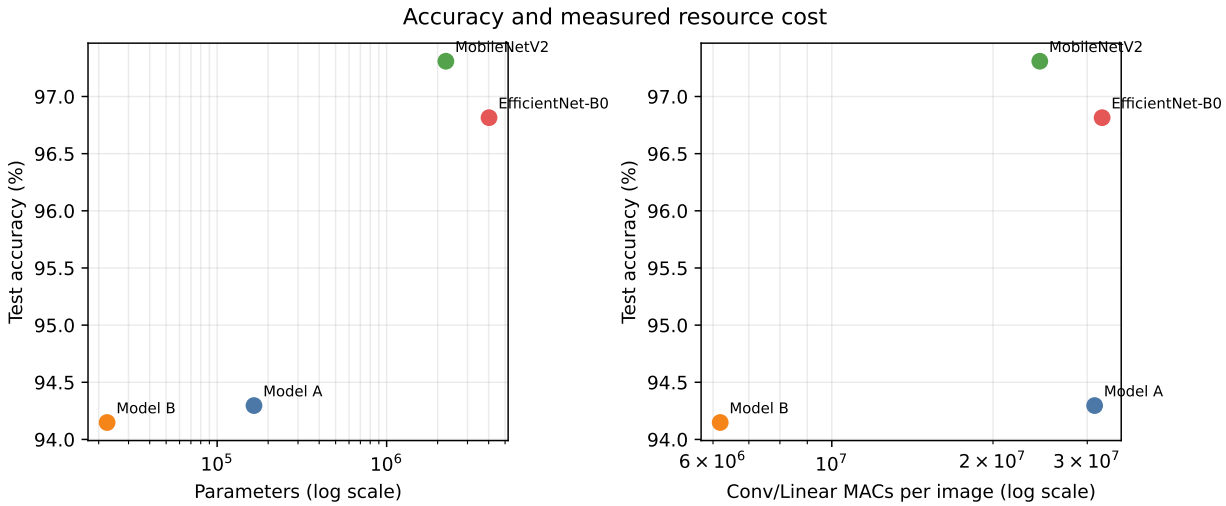


Figure 10: Held-out accuracy against parameters and Conv/Linear MACs. Horizontal axes use logarithmic scales.

7.5 Edge Deployment Trade-offs

Model B is a reasonable candidate when weight storage is the main constraint and its accuracy cost is acceptable. Model A remains competitive when the target resembles the measured host and latency matters more than checkpoint size: it is faster here and has slightly higher test accuracy. MobileNetV2 is the stronger pretrained candidate in these experiments because it combines higher

accuracy with lower storage and host latency than EfficientNet-B0. This conclusion is limited to the current split and schedule.

The pretrained models require more time per epoch, especially after unfreezing (Table 7). Their shorter fine-tuning schedules do not include the original ImageNet training cost. Neither MAC counts nor host measurements establish performance on an edge accelerator. Target-device operator support, quantization, peak RAM, latency and power should be checked before deployment. The present evidence establishes storage and arithmetic savings, not energy savings.

8 Discussion

The custom comparison holds channel widths, pooling schedule, training budget and optimizer fixed, while Model B introduces depthwise/pointwise factorization and additional normalization and ReLU operations. Thus the observed difference cannot be attributed solely to the convolution operator. Both custom models show validation fluctuations, making the saved best checkpoint preferable to blindly using the final epoch. The class-level errors explain why aggregate metrics should be read together. In Figure 4, 50 HerbaceousVegetation images are predicted as PermanentCrop, contributing to PermanentCrop precision of 82.46%. In Figure 5, 22 Highway images are predicted as Industrial; Highway recall is 86.67%. Model B thus changes the error distribution rather than uniformly reducing performance.

In Figure 7, the largest directed confusion count is 10, shared by PermanentCrop predicted as HerbaceousVegetation and River predicted as Highway. EfficientNet-B0's largest directed confusion is 14 AnnualCrop images predicted as PermanentCrop (Figure 9). Crop categories may share colour and texture at this resolution, but image-level analysis would be needed to establish the causes. The matrices show which errors occur, not why they occur.

Figure 11 shows two deterministic examples from each of four measured Model A confusion directions. The crop pairs support the matrix-level observation that some land-cover textures and long linear features can appear similar at 64×64. They are examples of recorded errors, not evidence that one visual feature caused every error.

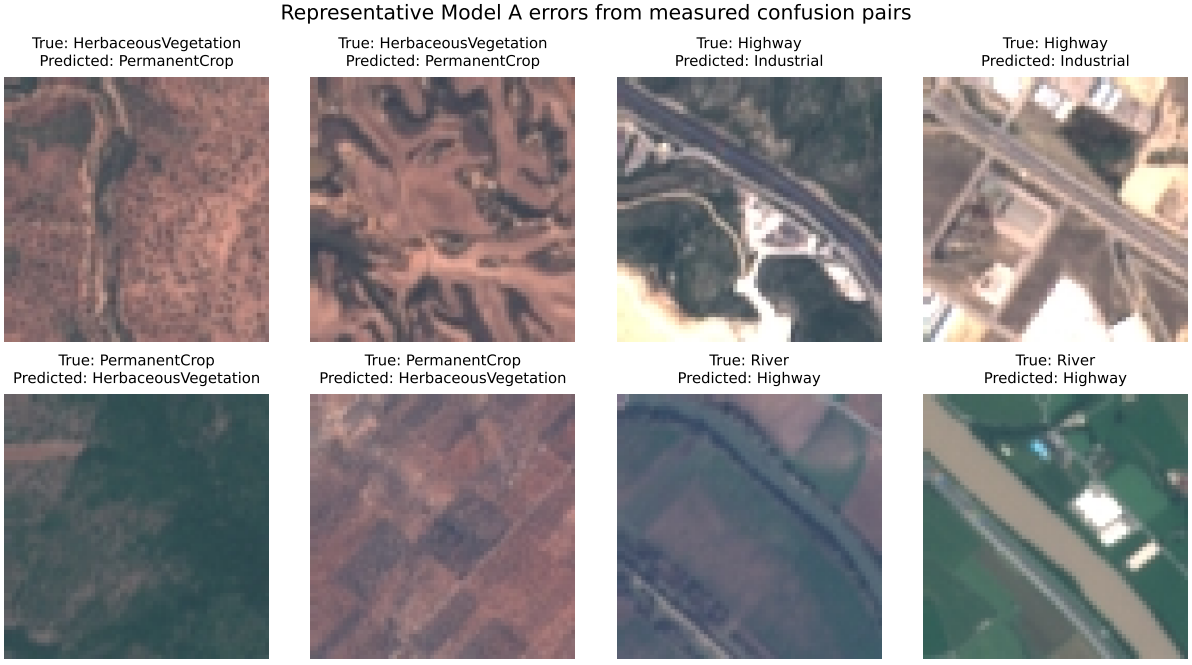


Figure 11: Representative Model A test errors selected from four confusion pairs.

These are single-seed observations with no confidence intervals or repeated-run variance estimate. Small accuracy differences should not be treated as proven general advantages. The split is stratified but not geographic; satellite scene correlation can make generalization to new regions harder than this test suggests. ImageNet transfer supplies additional prior information, and the 64x64 input differs from the pretrained recipe. These results are not the best attainable performance of the pretrained architectures. The optimizer pilot compares three fixed configurations for a short budget; it does not prove universal optimizer superiority. No unseen edge device or energy benefit is asserted.

The deployment choice is therefore conditional. Model B is attractive when weight storage is severe and a small accuracy loss is acceptable. MobileNetV2 is preferred when accuracy has priority and several MiB of weights are acceptable. EfficientNet-B0 is not preferred here because it is less accurate, larger and slower than MobileNetV2 under the tested setup. Before deployment, the team would still need target-device peak RAM, quantized accuracy, operator support, latency and power measurements.

9 Conclusion

The custom resource-constrained model meets the parameter requirement with 22,426 trainable parameters and 94.15% test accuracy. Relative to Model A, it saves 86.41% of parameters and 80.02% of counted MACs at an observed accuracy cost of 0.15 percentage points, although host timing does not improve. Momentum improves the tested SGD recipe, while Adam gives the strongest validation result and is used for both final custom runs. MobileNetV2 achieves the highest observed test accuracy, 97.31%, at a larger storage cost. The preferred model therefore depends on weight-storage capacity, acceptable accuracy loss and measured target-device performance, rather than parameter count alone. In this experiment EfficientNet-B0 does not offset its larger file and longer latency with higher accuracy than MobileNetV2. Actual edge hardware remains the final test because host latency, MACs and a single-activation estimate do not measure total runtime memory or energy.

References

[1] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.

[2] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[4] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[6] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 4510–4520, 2018.

[7] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 1139–1147, 2013.

[8] Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114, 2019.

A Layer-wise Architecture Tables

Appendix A provides the detailed layer-wise shapes and trainable parameter counts used to verify the two custom CNN architectures. DWConv and PWConv denote depthwise and pointwise convolutions, while GAP denotes global average pooling.

A.1 Model A

Table 14 lists the complete standard CNN and verifies its total trainable parameter count.

Table 14: Model A leaf layers, shapes and trainable parameter counts.

Layer	Operation	Input (NCHW)	Output	K/S/P	Params
f.0.0	Conv2d	$1 \times 3 \times 64 \times 64$	$1 \times 24 \times 64 \times 64$	3x3/1x1/1x1	648
f.0.1	BN	$1 \times 24 \times 64 \times 64$	$1 \times 24 \times 64 \times 64$	-/-/-	48
f.0.2	ReLU	$1 \times 24 \times 64 \times 64$	$1 \times 24 \times 64 \times 64$	-/-/-	0
f.0.3	MaxPool	$1 \times 24 \times 64 \times 64$	$1 \times 24 \times 32 \times 32$	2/2/0	0
f.1.0	Conv2d	$1 \times 24 \times 32 \times 32$	$1 \times 48 \times 32 \times 32$	3x3/1x1/1x1	10368
f.1.1	BN	$1 \times 48 \times 32 \times 32$	$1 \times 48 \times 32 \times 32$	-/-/-	96
f.1.2	ReLU	$1 \times 48 \times 32 \times 32$	$1 \times 48 \times 32 \times 32$	-/-/-	0
f.1.3	MaxPool	$1 \times 48 \times 32 \times 32$	$1 \times 48 \times 16 \times 16$	2/2/0	0
f.2.0	Conv2d	$1 \times 48 \times 16 \times 16$	$1 \times 96 \times 16 \times 16$	3x3/1x1/1x1	41472
f.2.1	BN	$1 \times 96 \times 16 \times 16$	$1 \times 96 \times 16 \times 16$	-/-/-	192
f.2.2	ReLU	$1 \times 96 \times 16 \times 16$	$1 \times 96 \times 16 \times 16$	-/-/-	0
f.2.3	MaxPool	$1 \times 96 \times 16 \times 16$	$1 \times 96 \times 8 \times 8$	2/2/0	0
f.3.0	Conv2d	$1 \times 96 \times 8 \times 8$	$1 \times 128 \times 8 \times 8$	3x3/1x1/1x1	110592
f.3.1	BN	$1 \times 128 \times 8 \times 8$	$1 \times 128 \times 8 \times 8$	-/-/-	256
f.3.2	ReLU	$1 \times 128 \times 8 \times 8$	$1 \times 128 \times 8 \times 8$	-/-/-	0
f.3.3	MaxPool	$1 \times 128 \times 8 \times 8$	$1 \times 128 \times 4 \times 4$	2/2/0	0
pool	GAP	$1 \times 128 \times 4 \times 4$	$1 \times 128 \times 1 \times 1$	-/-/-	0
flatten	Flatten	$1 \times 128 \times 1 \times 1$	1×128	-/-/-	0
head	Linear	1×128	1×10	-/-/-	1290
				Total trainable parameters	164,962

A.2 Model B

Table 15 lists the complete depthwise-separable CNN and independently verifies that Model B remains below the 100,000-parameter limit.

Table 15: Model B leaf layers, shapes and trainable parameter counts.

Layer	Operation	Input (NCHW)	Output	K/S/P	Params
f.0.0	Conv2d	$1 \times 3 \times 64 \times 64$	$1 \times 24 \times 64 \times 64$	3x3/1x1/1x1	648
f.0.1	BN	$1 \times 24 \times 64 \times 64$	$1 \times 24 \times 64 \times 64$	-/-/-	48
f.0.2	ReLU	$1 \times 24 \times 64 \times 64$	$1 \times 24 \times 64 \times 64$	-/-/-	0
f.0.3	MaxPool	$1 \times 24 \times 64 \times 64$	$1 \times 24 \times 32 \times 32$	2/2/0	0
f.1.0.0	DWConv	$1 \times 24 \times 32 \times 32$	$1 \times 24 \times 32 \times 32$	3x3/1x1/1x1	216
f.1.0.1	BN	$1 \times 24 \times 32 \times 32$	$1 \times 24 \times 32 \times 32$	-/-/-	48
f.1.0.2	ReLU	$1 \times 24 \times 32 \times 32$	$1 \times 24 \times 32 \times 32$	-/-/-	0
f.1.0.3	PWConv	$1 \times 24 \times 32 \times 32$	$1 \times 48 \times 32 \times 32$	1x1/1x1/0x0	1152
f.1.0.4	BN	$1 \times 48 \times 32 \times 32$	$1 \times 48 \times 32 \times 32$	-/-/-	96
f.1.0.5	ReLU	$1 \times 48 \times 32 \times 32$	$1 \times 48 \times 32 \times 32$	-/-/-	0
f.1.1	MaxPool	$1 \times 48 \times 32 \times 32$	$1 \times 48 \times 16 \times 16$	2/2/0	0
f.2.0.0	DWConv	$1 \times 48 \times 16 \times 16$	$1 \times 48 \times 16 \times 16$	3x3/1x1/1x1	432
f.2.0.1	BN	$1 \times 48 \times 16 \times 16$	$1 \times 48 \times 16 \times 16$	-/-/-	96
f.2.0.2	ReLU	$1 \times 48 \times 16 \times 16$	$1 \times 48 \times 16 \times 16$	-/-/-	0
f.2.0.3	PWConv	$1 \times 48 \times 16 \times 16$	$1 \times 96 \times 16 \times 16$	1x1/1x1/0x0	4608
f.2.0.4	BN	$1 \times 96 \times 16 \times 16$	$1 \times 96 \times 16 \times 16$	-/-/-	192
f.2.0.5	ReLU	$1 \times 96 \times 16 \times 16$	$1 \times 96 \times 16 \times 16$	-/-/-	0
f.2.1	MaxPool	$1 \times 96 \times 16 \times 16$	$1 \times 96 \times 8 \times 8$	2/2/0	0
f.3.0.0	DWConv	$1 \times 96 \times 8 \times 8$	$1 \times 96 \times 8 \times 8$	3x3/1x1/1x1	864
f.3.0.1	BN	$1 \times 96 \times 8 \times 8$	$1 \times 96 \times 8 \times 8$	-/-/-	192
f.3.0.2	ReLU	$1 \times 96 \times 8 \times 8$	$1 \times 96 \times 8 \times 8$	-/-/-	0
f.3.0.3	PWConv	$1 \times 96 \times 8 \times 8$	$1 \times 128 \times 8 \times 8$	1x1/1x1/0x0	12288
f.3.0.4	BN	$1 \times 128 \times 8 \times 8$	$1 \times 128 \times 8 \times 8$	-/-/-	256
f.3.0.5	ReLU	$1 \times 128 \times 8 \times 8$	$1 \times 128 \times 8 \times 8$	-/-/-	0
f.3.1	MaxPool	$1 \times 128 \times 8 \times 8$	$1 \times 128 \times 4 \times 4$	2/2/0	0
pool	GAP	$1 \times 128 \times 4 \times 4$	$1 \times 128 \times 1 \times 1$	-/-/-	0
flatten	Flatten	$1 \times 128 \times 1 \times 1$	1×128	-/-/-	0
head	Linear	1×128	1×10	-/-/-	1290
				Total trainable parameters	22,426

B Reproducibility

All experiments use seed 3150 and the same saved stratified train/validation/test split. Model and optimizer selection use only validation performance, while the held-out test set is used for final evaluation. The reported metrics and confusion matrices were cross-checked against the saved experiment outputs. The same split is used for all four models to keep the comparison fair.